Case 2: Market Segmentation Using Clustering

Mohammed Baobaid - 202031137

2026-01-28

0. Setup & Reproducibility

0.1 Install Required Packages (Run Once)

```
required_packages <- c(
  "tidyverse",
  "lubridate",
  "forecast",
  "tseries"
)

installed <- rownames(installed.packages())

for (pkg in required_packages) {
  if (!(pkg %in% installed)) {
    install.packages(pkg)
  }
}
```

0.2 Load Libraries & Set Seed

```
SEED <- 202031137
set.seed(SEED)

library(tidyverse)
library(lubridate)
library(forecast)
library(tseries)

theme_set(theme_minimal())
```

1. Data Loading and Initial Inspection

1.1 Load Dataset

```r
file_path <- "Datasets/train.csv"

if (!file.exists(file_path)) {
  stop("File not found. Please check dataset path.")
}

train <- read_csv(file_path, show_col_types = FALSE)

head(train)
```

```
## # A tibble: 6 x 4
##   date       store  item sales
##   <date>     <dbl> <dbl> <dbl>
## 1 2013-01-01     1     1    13
## 2 2013-01-02     1     1    11
## 3 2013-01-03     1     1    14
## 4 2013-01-04     1     1    13
## 5 2013-01-05     1     1    10
## 6 2013-01-06     1     1    12
```

1.2 Structure & Quality Check

```r
glimpse(train)
```

```
## Rows: 913,000
## Columns: 4
## $ date  <date> 2013-01-01, 2013-01-02, 2013-01-03, 2013-01-04, 2013-01-05, 201~
## $ store <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
## $ item  <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
## $ sales <dbl> 13, 11, 14, 13, 10, 12, 10, 9, 12, 9, 9, 7, 10, 12, 5, 7, 16, 7,~
```

```r
summary(train)
```

```
##       date                 store           item          sales       
##  Min.   :2013-01-01   Min.   : 1.0   Min.   : 1.0   Min.   :  0.00  
##  1st Qu.:2014-04-02   1st Qu.: 3.0   1st Qu.:13.0   1st Qu.: 30.00  
##  Median :2015-07-02   Median : 5.5   Median :25.5   Median : 47.00  
##  Mean   :2015-07-02   Mean   : 5.5   Mean   :25.5   Mean   : 52.25  
##  3rd Qu.:2016-10-01   3rd Qu.: 8.0   3rd Qu.:38.0   3rd Qu.: 70.00  
##  Max.   :2017-12-31   Max.   :10.0   Max.   :50.0   Max.   :231.00
```

```r
colSums(is.na(train))
```

```
##  date store  item sales 
##     0     0     0     0
```

2. Data Preparation

2.1 Convert Date Column

```
train <- train %>%
  mutate(date = as.Date(date, format = "%d/%m/%Y"))
```

2.2 Filter to Single Store & Item

```
train_filtered <- train %>%
  filter(store == 1, item == 1)

nrow(train_filtered)
```

```
## [1] 1826
```

2.3 Aggregate Daily to Monthly Sales

```
monthly_sales <- train_filtered %>%
  mutate(month = floor_date(date, "month")) %>%
  group_by(month) %>%
  summarise(monthly_sales = sum(sales), .groups = "drop")

head(monthly_sales)
```

```
## # A tibble: 6 x 2
##   month      monthly_sales
##   <date>             <dbl>
## 1 2013-01-01           328
## 2 2013-02-01           322
## 3 2013-03-01           477
## 4 2013-04-01           522
## 5 2013-05-01           531
## 6 2013-06-01           627
```

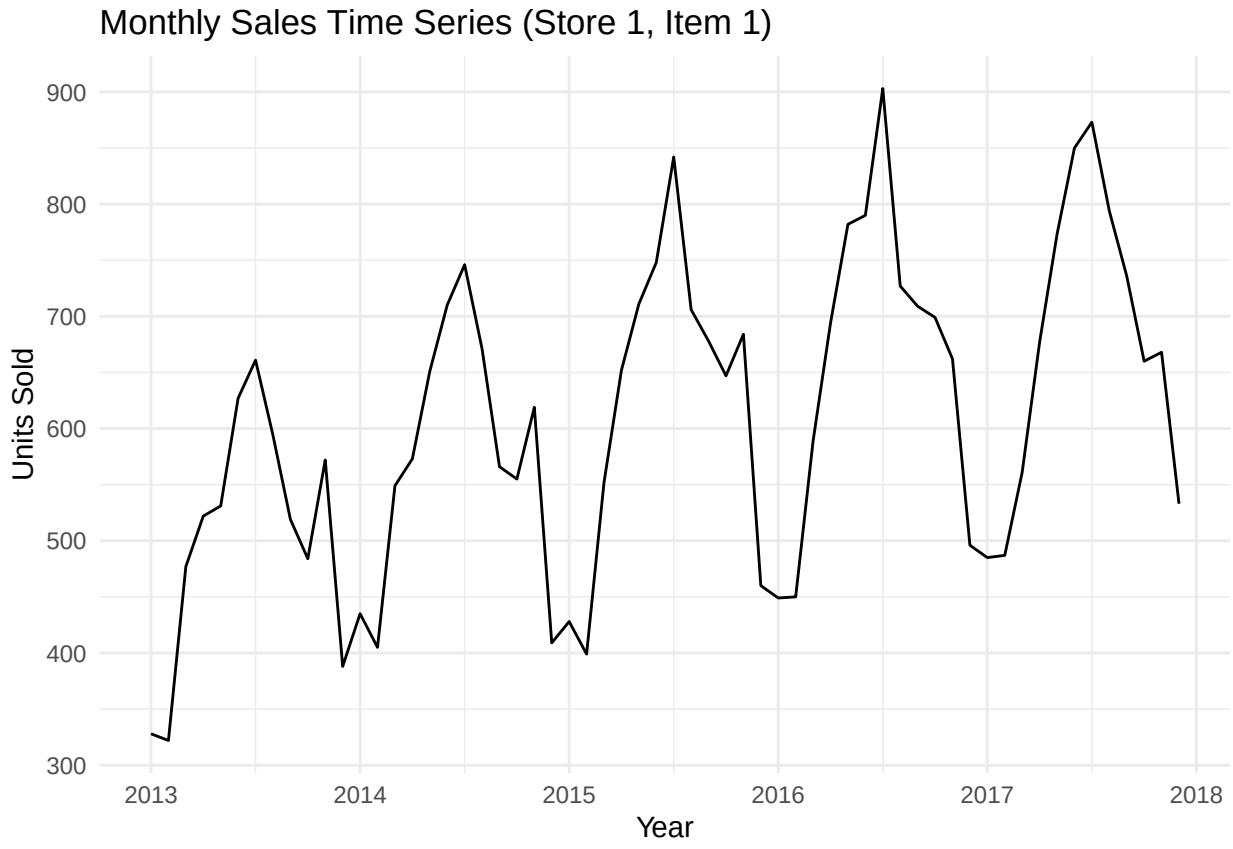
2.4 Create Time Series Object

```
sales_ts <- ts(
  monthly_sales$monthly_sales,
  start = c(year(min(monthly_sales$month)),
            month(min(monthly_sales$month))),
  frequency = 12
)

sales_ts
```

```
##      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
## 2013 328 322 477 522 531 627 661 594 519 484 572 388
## 2014 435 405 549 573 651 710 746 671 566 555 619 409
## 2015 428 399 552 652 711 748 842 706 678 647 684 460
## 2016 449 450 589 694 782 790 903 727 709 699 662 496
## 2017 485 487 561 677 773 850 873 794 736 660 668 533
```

2.5 Initial Time Series Plot

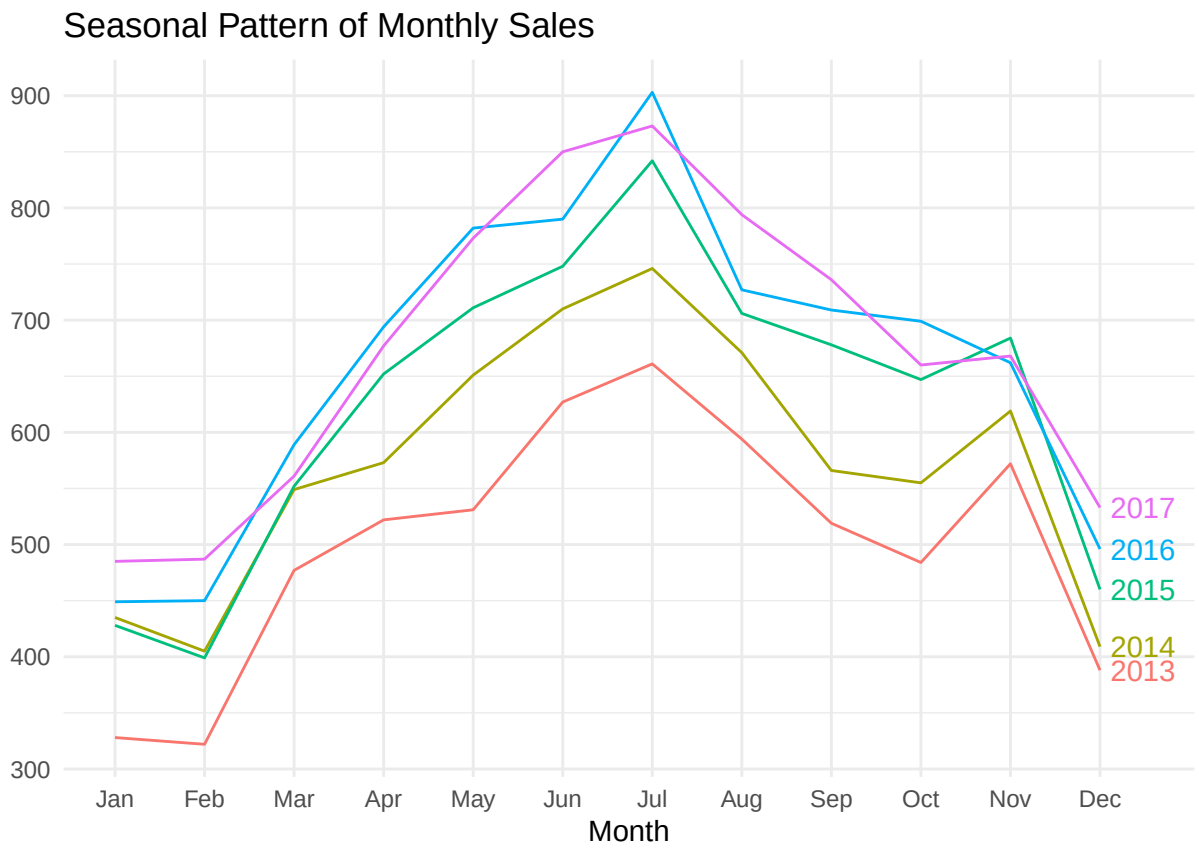
```
autoplot(sales_ts) +
  ggtitle("Monthly Sales Time Series (Store 1, Item 1)") +
  xlab("Year") +
  ylab("Units Sold")
```



3. Exploratory Time Series Analysis

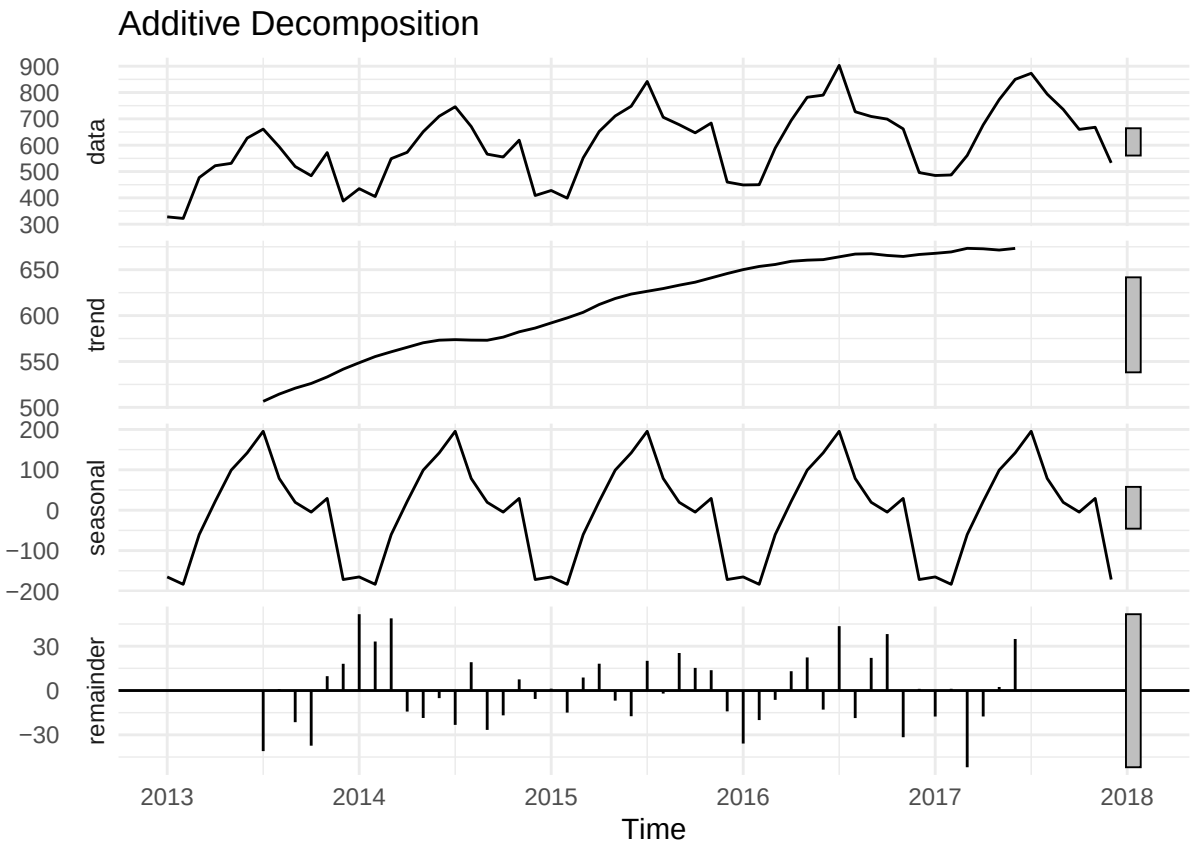
3.1 Seasonal Pattern

```
ggseasonplot(sales_ts, year.labels = TRUE) +
  ggtitle("Seasonal Pattern of Monthly Sales")
```

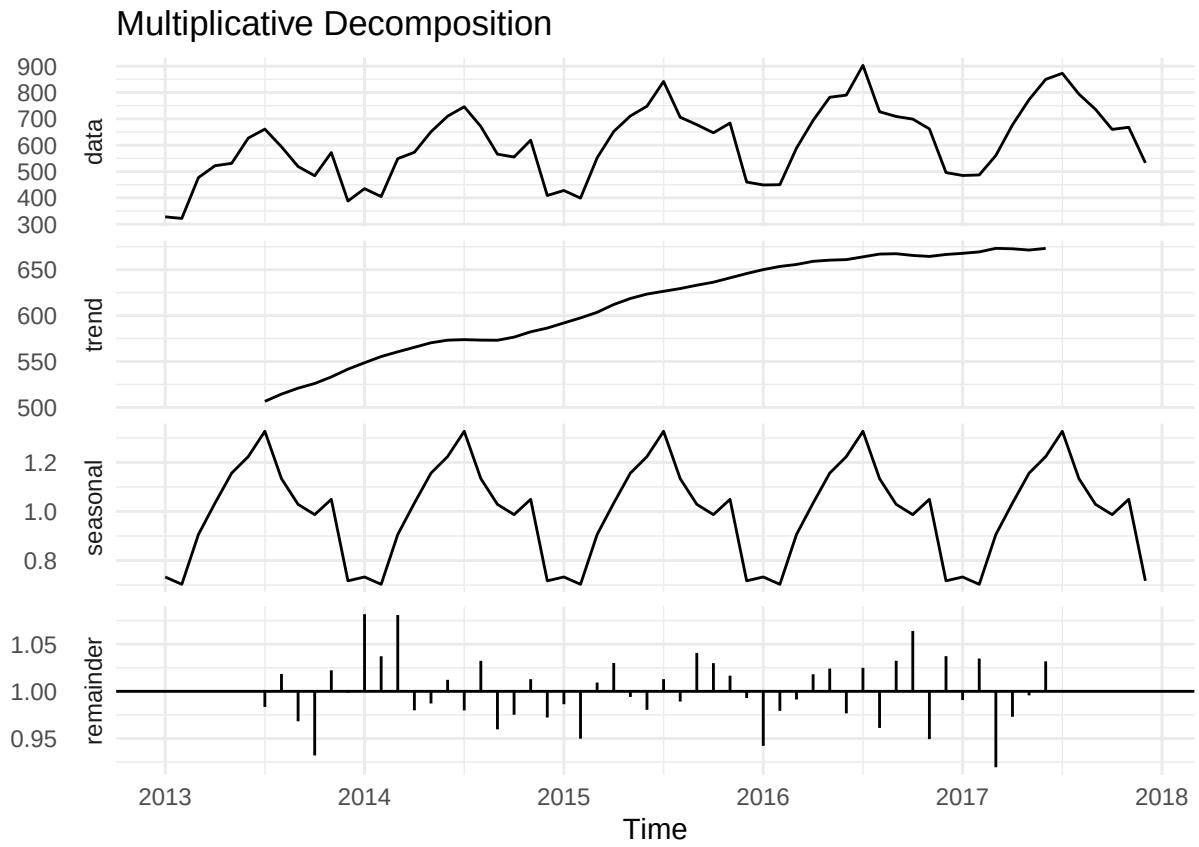


3.2 Decomposition

```
decomp_add <- decompose(sales_ts, type = "additive")
decomp_mult <- decompose(sales_ts, type = "multiplicative")

autoplot(decomp_add) + ggtitle("Additive Decomposition")
```




```
autoplot(decomp_mult) + ggtitle("Multiplicative Decomposition")
```



3.3 Compare Decomposition Fit

```
sd(decomp_add$random, na.rm = TRUE)
```

```
## [1] 24.23664
```

```
sd(decomp_mult$random, na.rm = TRUE)
```

```
## [1] 0.03572288
```

4. Stationarity Testing

4.1 ADF Test

```
adf.test(sales_ts)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  sales_ts
## Dickey-Fuller = -4.9677, Lag order = 3, p-value = 0.01
## alternative hypothesis: stationary
```

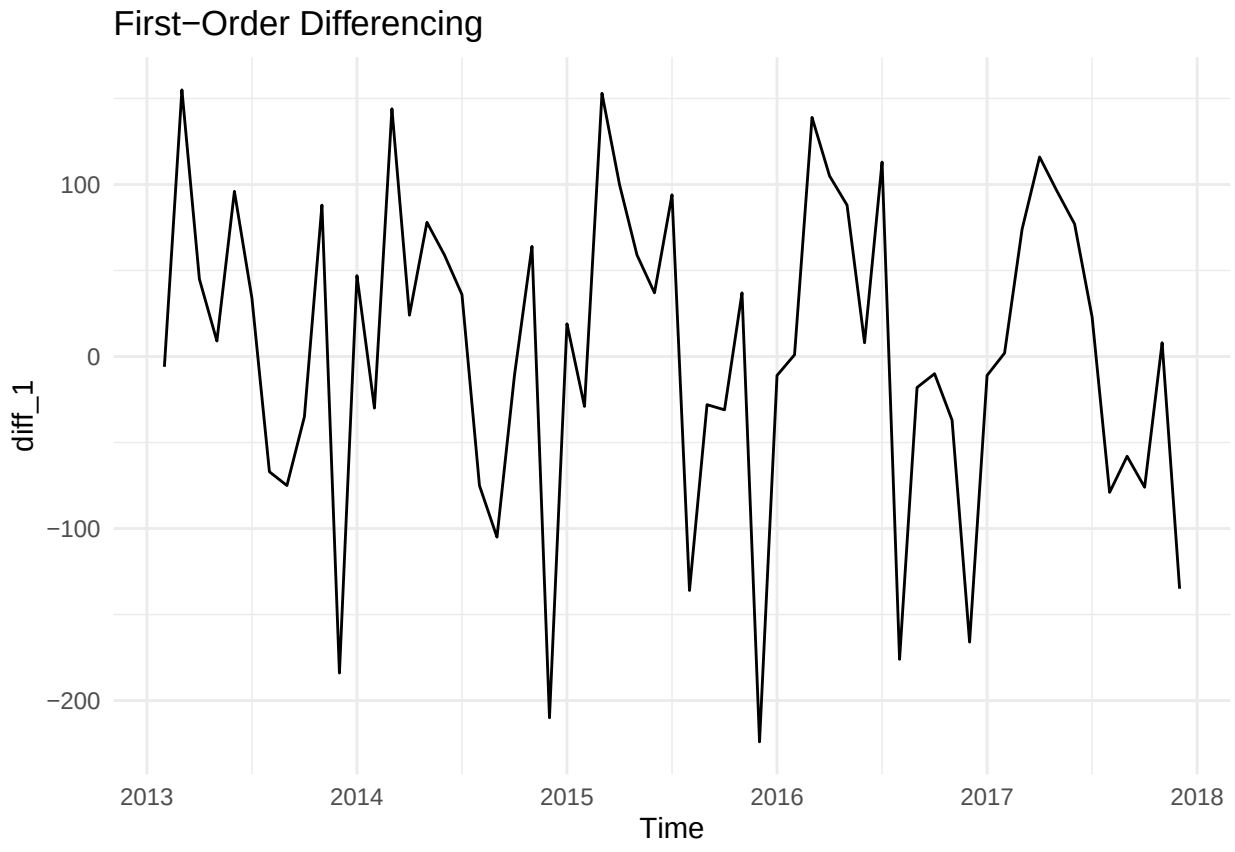
4.2 KPSS Test

```
kpss.test(sales_ts)
```

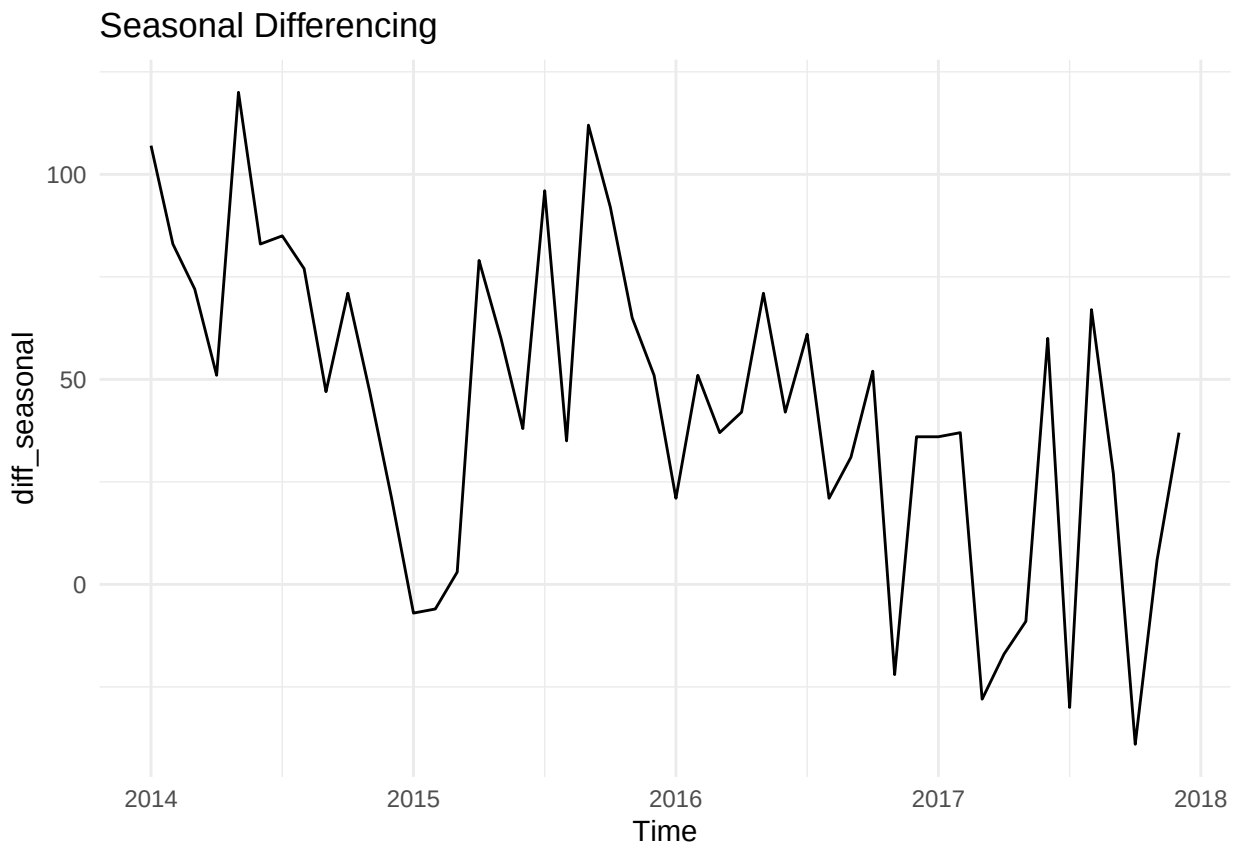
```
##
##  KPSS Test for Level Stationarity
##
## data:  sales_ts
## KPSS Level = 0.548, Truncation lag parameter = 3, p-value = 0.03086
```

4.3 Differencing

```
diff_1 <- diff(sales_ts, differences = 1)
autoplot(diff_1) + ggtitle("First-Order Differencing")
```




```
diff_seasonal <- diff(sales_ts, lag = 12)
autoplot(diff_seasonal) + ggtitle("Seasonal Differencing")
```



5. Model Development

5.1 ARIMA Model

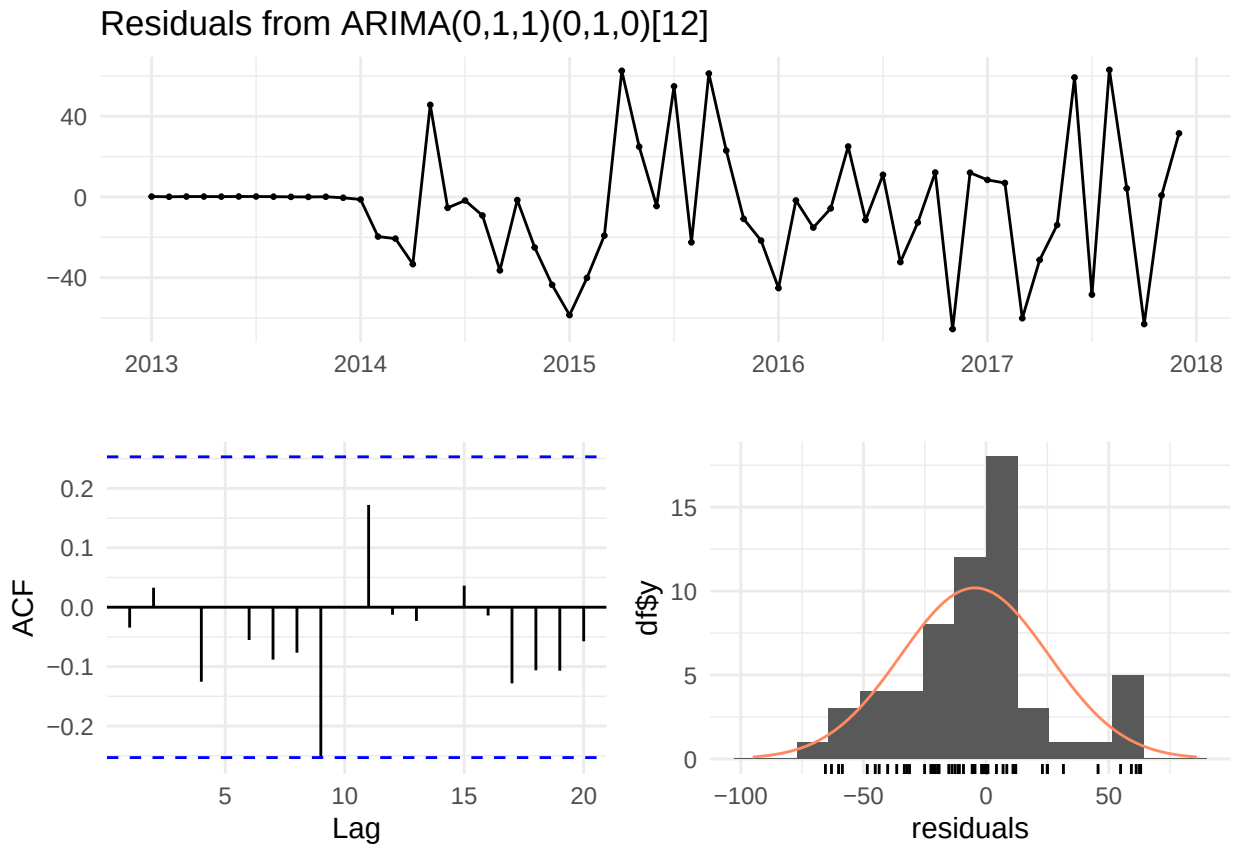
```
arima_model <- auto.arima(
  sales_ts,
  seasonal = TRUE,
  stepwise = FALSE,
  approximation = FALSE
)

summary(arima_model)
```

```
## Series: sales_ts
## ARIMA(0,1,1)(0,1,0)[12]
##
## Coefficients:
##          ma1
##       -0.702
## s.e.   0.125
##
## sigma^2 = 1196:  log likelihood = -233.06
```

```
## AIC=470.11   AICc=470.39   BIC=473.81
##
## Training set error measures:
##                     ME     RMSE      MAE       MPE     MAPE      MASE
## Training set -4.574964 30.27817 21.50033 -1.114401 3.538521 0.4321674
##                     ACF1
## Training set -0.03428181
```

5.2 Residual Diagnostics

```
checkresiduals(arima_model)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)(0,1,0)[12]
## Q* = 9.3756, df = 11, p-value = 0.5873
##
## Model df: 1.   Total lags used: 12
```

5.3 ETS Model

```
ets_model <- ets(sales_ts)
summary(ets_model)
```

```
## ETS(M,Ad,M)
##
## Call:
## ets(y = sales_ts)
##
##   Smoothing parameters:
##     alpha = 0.0284
##     beta  = 0.001
##     gamma = 1e-04
##     phi   = 0.9789
##
##   Initial states:
##     l = 481.562
##     b = 6.6275
##     s = 0.725 1.0326 0.9817 1.035 1.1383 1.3203
##            1.2282 1.1522 1.0397 0.9263 0.6971 0.7234
##
##   sigma:  0.0506
##
##      AIC     AICc      BIC
## 670.2209 686.9039 707.9191
##
## Training set error measures:
##                     ME     RMSE      MAE       MPE     MAPE      MASE
## Training set -3.121402 24.89179 20.58181 -0.765493 3.555211 0.4137047
##                     ACF1
## Training set 0.005030704
```

5.4 Model Comparison

```
accuracy(arima_model)
```

```
##                     ME     RMSE      MAE       MPE     MAPE      MASE
## Training set -4.574964 30.27817 21.50033 -1.114401 3.538521 0.4321674
##                     ACF1
## Training set -0.03428181
```

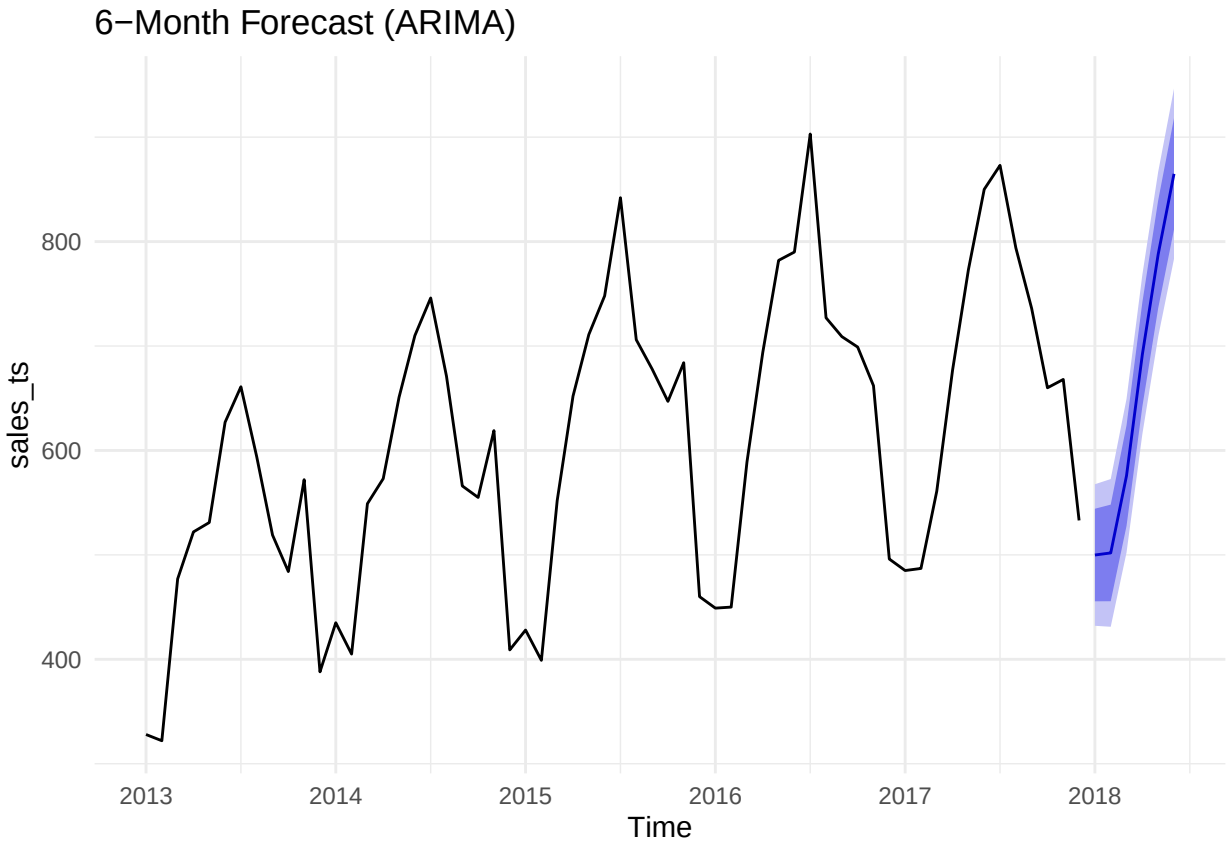
```
accuracy(ets_model)
```

```
##                     ME     RMSE      MAE       MPE     MAPE      MASE
## Training set -3.121402 24.89179 20.58181 -0.765493 3.555211 0.4137047
##                     ACF1
## Training set 0.005030704
```

6. Forecasting

6.1 ARIMA Forecast

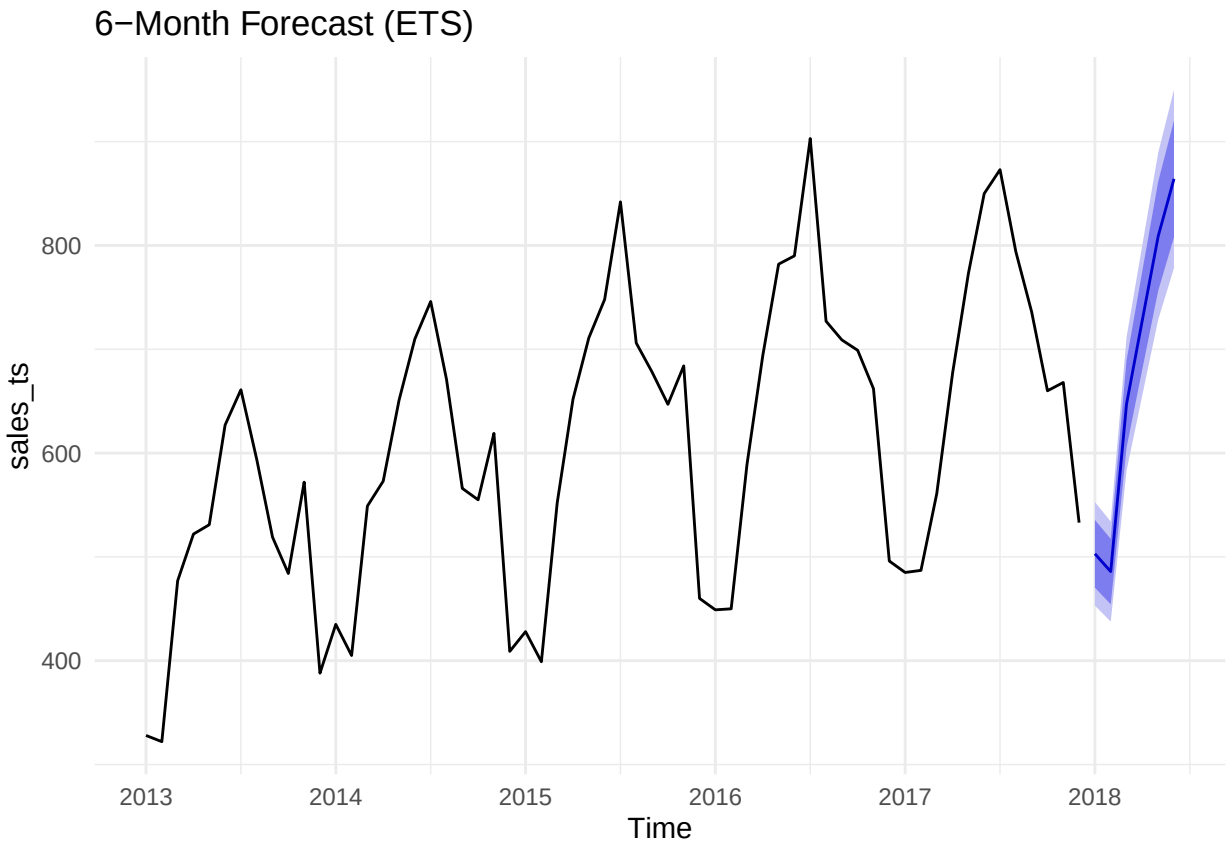
```
arima_forecast <- forecast(
  arima_model,
  h = 6,
  level = c(80, 95)
)

autoplot(arima_forecast) +
  ggtitle("6-Month Forecast (ARIMA)")
```



6.2 ETS Forecast

```
ets_forecast <- forecast(
  ets_model,
  h = 6,
  level = c(80, 95)
)
```

```r
autoplot(ets_forecast) +
  ggtitle("6-Month Forecast (ETS)")
```



6.3 Forecast Table

```r
forecast_table <- data.frame(
  Month = time(arima_forecast$mean),
  Forecast = as.numeric(arima_forecast$mean),
  Lower_80 = arima_forecast$lower[,1],
  Upper_80 = arima_forecast$upper[,1],
  Lower_95 = arima_forecast$lower[,2],
  Upper_95 = arima_forecast$upper[,2]
)

forecast_table
```

```
##      Month Forecast Lower_80 Upper_80 Lower_95 Upper_95
## 1 2018.000 499.8668 455.5506 544.1830 432.0910 567.6426
## 2 2018.083 501.8668 455.6244 548.1092 431.1451 572.5885
## 3 2018.167 575.8668 527.7753 623.9583 502.3171 649.4165
## 4 2018.250 691.8668 641.9947 741.7389 615.5939 768.1397
## 5 2018.333 787.8668 736.2755 839.4581 708.9647 866.7690
## 6 2018.417 864.8668 811.6117 918.1219 783.4202 946.3134
```

7. Model Evaluation

7.1 Train-Test Split

```
train_ts <- head(sales_ts, length(sales_ts) - 6)
test_ts  <- tail(sales_ts, 6)
```

7.2 Refit Models

```
arima_train <- auto.arima(train_ts)
ets_train   <- ets(train_ts)
```

7.3 Forecast Test Period

```
arima_test_forecast <- forecast(arima_train, h = 6)
ets_test_forecast   <- forecast(ets_train, h = 6)
```

7.4 Accuracy Metrics

```
accuracy(arima_test_forecast, test_ts)
```

```
##                      ME     RMSE      MAE        MPE     MAPE      MASE
## Training set  -3.582834 28.34954 19.81777 -0.9757333 3.325648 0.3814602
## Test set     -10.108952 38.48480 32.33333 -1.3250768 4.462333 0.6223648
##                     ACF1 Theil's U
## Training set  0.004045687        NA
## Test set     -0.252108515 0.4033977
```

```
accuracy(ets_test_forecast, test_ts)
```

```
##                     ME     RMSE      MAE        MPE     MAPE      MASE
## Training set -2.28354 23.92564 19.12915 -0.6516379 3.376355 0.3682054
## Test set     -5.32400 35.18759 32.60035 -0.5006034 4.771792 0.6275044
##                    ACF1 Theil's U
## Training set  0.05277298        NA
## Test set     -0.24931145 0.4468858
```

```
actual <- as.numeric(test_ts)

predicted_arima <- as.numeric(arima_test_forecast$mean)
mape_arima <- mean(abs((actual - predicted_arima) / actual)) * 100
rmse_arima <- sqrt(mean((actual - predicted_arima)^2))

predicted_ets <- as.numeric(ets_test_forecast$mean)
```

```r
mape_ets <- mean(abs((actual - predicted_ets) / actual)) * 100
rmse_ets <- sqrt(mean((actual - predicted_ets)^2))

mape_arima
```

```
## [1] 4.462333
```

```r
rmse_arima
```

```
## [1] 38.4848
```

```r
mape_ets
```

```
## [1] 4.771792
```

```r
rmse_ets
```

```
## [1] 35.18759
```